СОГЛАСОВАННАЯ АРХИТЕКТУРА

ML: реальные функции в компактном MVP

Единый контракт, проверяемые границы и конкретные правила реализации.

20 октября 2026
Согласованная спецификация

Команда 3-5 человек
Окно 48-72 часа

Приложение не реализовано. Статические проверки документации и контрактов отделены от будущих испытаний runtime.

Связанные источники, контракты и приёмка доступны в Markdown-руководстве и общем index.html комплекта.

HACKATHON-1.5. ML обязателен в согласованном объёме: риск, semantic-рекомендации, OCR и RAG. Наличие этой спецификации не означает, что веса скачаны, модели обучены или качество измерено. Промышленные правила прежнего ML-пакета сохранены в contracts/bank-target-release-policy.json и архиве, но не являются условиями локального canary на синтетическом трафике.

1. Минимальный runtime

Python 3.12 + FastAPI/Pydantic 2, NumPy/scikit-learn для риска, multilingual-e5-small для embeddings, Tesseract 5.x/tessdata_fast rus+eng для OCR, Qwen2.5-0.5B-Instruct-GGUF Q4_K_M в llama-server. Конкретные версии Python-зависимостей, revision моделей, traineddata и OCI digests фиксируются P02/P11-P14. Пока manifest не содержит реальные SHA-256, релиз не допускается; фиктивных hashes здесь нет.

Qwen выбран как малый открытый CPU baseline, а не как доказанно качественная банковская модель. E5 используется и для текстовой близости продуктов, и для поиска знаний с отдельными индексами/версиями. Для малого корпуса достаточно нормализованной матрицы NumPy и metadata. Runtime не запускает training и не скачивает модели. Точный NVIDIA GPU/VRAM неизвестен; GPU ускоряет тот же bundle и не меняет модель автоматически.

Проектный ориентир CPU-стенда: 8 logical cores, 16 GiB RAM, 20 GiB свободного диска. Это начальная конфигурация для измерения, не подтверждённый минимальный размер. Проверяется поддерживаемый CPU ISA и архитектура OCI; arm64/amd64 - отдельные smoke. GPU-profile только после проверки driver/CUDA и паритета результатов.

2. Риск: смысл, данные, обучение

Актуальный первый скоринговый срез от 21.10.2026 использует полностью вымышленные компании, финансовые суммы и target_12m из training_simulation.parquet. Настоящие реквизиты гибридной таблицы не используются. Прежние UCI 5year.arff и grouped split 60/20/20 для этого релиза заменены готовым company-disjoint split поставки: train 7016/352 событий, validation 1494/80, test 1490/80. Проверяются опубликованные SHA-256; разбиение не пересоздаётся.

Реализован CatBoostClassifier с подбором гиперпараметров Optuna. Общий batch/online builder рассчитывает X1=netProfit/assets, X2=liabilities/assets, X3=(currentAssets-currentLiabilities)/assets, X4=currentAssets/currentLiabilities, X7=EBIT/assets, X8=equity/liabilities, X9=revenue/assets, X10=equity/assets. Деньги поступают как целые копейки, Decimal-конвертация из Parquet исключает промежуточный float. Баланс проверяется с допуском 1 копейка. При assets≤0 оценка невозможна; нулевые знаменатели дают missing. Пропуски CatBoost обрабатывает через nan_mode=Min, без imputer/scaler.

Optuna TPE(seed=42) выполняет 30 trials с трёхкратной CV на train и MedianPruner. Подбираются глубина/деревья/learning rate/регуляризация/случайность/выборка признаков/границы/веса классов. Пять лучших trials оцениваются на validation; версия выбирается по validation AP. Модель и policy фиксируются до открытия test. Сценарии выбираются только из train. Target, IDs, split, даты, технические флаги и simulation_truth не являются признаками. История подбора сохранена в optuna-study.json.

T_manual выбран на validation как верхние 20% score пригодных наблюдений. Зафиксировано 0.03504223273828764; пригодных 1205, manual 241; 26 искусственных событий в manual. Это демонстрационный бюджет проверки. На test ROC-AUC 0.90449, AP 0.40560; полные prevalence/Brier/log loss/confusion/distribution и hashes лежат в reports. Все метрики относятся только к искусственной симуляции.

Срез включает ML API, S01–S06 и эталон backend policy. Команды и интеграция описаны в docs/ml-integration.md корня проекта. UI/backend и прочие ML-функции ниже сохраняют проектный статус.

3. Неизменяемые признаки и inference

Java фиксирует финансовый снимок; feature-builder читает его, вычисляет отношения и сохраняет FeatureSnapshot в ml-схеме с featureSchemaVersion=2, code digest, asOf, source availability и digest. Владелец feature snapshot API - ML; вычисления выполняет общий Python-transform. Bank adapter передаёт финансовые значения из неизменяемого CreditPackage в createFeatureSnapshot, не давая Python доступ к текущему черновику. Повтор одного submissionSnapshotId/codeDigest возвращает тот же снимок.

Risk API получает snapshotId/digest и читает разрешённый snapshot через GET /internal/v2/feature-snapshots/{featureSnapshotId}. Не пересчитывать из текущего черновика. Время availableAt каждого входа не позже asOf; исторические исправления не переносятся в прошлое. Hash считается от канонического JSON с фиксированными именами/порядком сериализации; версии формата входят в manifest.

Ответ BANKRUPTCY_PROXY/DEMONSTRATION связывается с requestId, inferenceId, featureSnapshotDigest, modelVersion и bundleDigest. Inference record должен быть долговечным до успешного ответа; недоступный внешний сборщик метрик не равен отказу локального журнала. Score и internalReasonCodes видит банковский симулятор, но не клиентское API.

Бюджет risk 3 с; повтор с тем же requestId возможен только для того же snapshot/bundle. Ошибка, несовместимая схема или невозможность сохранить inference record → MANUAL. Риск не изменяет решение или offer самостоятельно. PD/VALIDATED и purpose=CREDIT_RISK закрыты в local-mvp независимо от того, что общая schema описывает target-возможность.

4. Рекомендации как содержательная модель

Пользовательские поведенческие метки пока отсутствуют. Поэтому реальная ML-функция MVP - pretrained semantic ranking E5, без заявления об обученной по кликам персонализации. Для PRODUCT контекст - назначение финансирования и coarse business category; для FEED - стадия и утверждённый текст полезного следующего шага. Персональные финансы/причины риска/документы не передаются ranker.

Backend формирует пригодные candidateIds/candidateTexts, catalogVersion и eligibilityVersion. Python применяет документированные E5 префиксы query:/passage:, mean pooling с attention mask и L2 normalization; считает cosine similarity. Tie-break по исходному стабильному порядку. Ответ содержит ровно перестановку кандидатов. Модель не принимает решение о пригодности и не генерирует условия продукта.

PRODUCT и FEED имеют отдельные artifact manifests и кеши; source model может быть общим. Изменение catalog/text/model revision требует нового digest. Embeddings кандидатов готовятся до показа; прогрев query encoder входит в readiness. Бюджет ranking 2 с. При ошибке → RULES с тем же допустимым набором; pinned next action не входит в перестановку.

Exposure записывает сервер. Frontend сообщает реальные VIEW/CLICK с дедупликацией; completion/submit приходят из бизнес-событий. Без закрытых окон атрибуции и достаточных меток не запускать CatBoostRanker и не оптимизировать выдачу кредита по CTR.

5. OCR одной формы

Поддерживается собственная DemoFinancial v1: одна страница, восемь полей, явные подписи, год, RUB и масштаб «рубли с копейками». Исходные synthetic PNG/PDF и варианты печати включаются в будущий seed. Это не универсальное распознавание отчётности РСБУ/МСФО.

Tesseract rus+eng извлекает текст и координаты; fixed-layout parser нормализует пробелы, знак, десятичный разделитель и minor units. Нечитаемое значение остаётся missing; знак/баланс не «чинится» догадкой. Confidence нормализуется в [0,1] из engine score и помечается как

техническая величина, а не вероятность достоверности поля.

Worker получает только CLEAN objectVersion и действующую lease; результат содержит generation, worker/token, engineVersion, поля/страницы. Парсер-версия и traineddata hashes входят в bundle. Любой результат требует подтверждения пользователя; correction сохраняется рядом с исходным OCR и точной версией файла. При отказе OCR ручной ввод сохраняет основной путь.

Проверка MVP: минимум 10 synthetic вариантов, включая отрицательную прибыль, нулевой долг, знак минус, запятую/пробелы и низкое качество. Измерять exact match восьми полей после нормализации и ошибки знака/масштаба. Эти примеры не доказывают промышленную точность. Target-набор 200 проверенных документов и canary оставлен следующим этапом.

6. RAG и публикация знаний

Начальный корпус - 15-30 коротких утверждённых synthetic статей о процессе и demo-продуктах. Компания/полномочия/consent сохраняются в backend. Статусы знаний DRAFT/PUBLISHED/REVOKED, documentVersion, effectiveUntil и audience задаёт KB authority. Локальная публикация через служебный API требует отдельной роли content-owner; клиент не загружает документы в KB через OCR.

Ingestion ограничивает размер, очищает активное содержимое, разбивает текст примерно по 300 токенов с overlap40 и сохраняет IDs/chunk hashes. Авторизованный corpus manifest задаёт версии. E5 retrieval top3, точные права/публикация проверяются до помещения в prompt. Threshold подбирается на сохранённом QA-наборе P14: точное значение и evaluation digest обязательны в bundle, универсального similarity threshold нет.

Генерация: Qwen GGUF Q4_K_M, context2048 tokens, output≤192 tokens, concurrency1, temperature0.1. Бюджет генерации 45 с внутри общего 60 с; max очередь 4, 1 запрос/сессию и 5 запросов/мин. Превышение очереди →429. Перед вызовом исключить sensitive финансовые данные и внутренние причины решения. Модель не имеет tools, сетевого доступа и прав записи в заявку.

Ответ ANSWER содержит внутренние ссылки на использованные document/version. Backend проверяет, что все citations входят в retrieval allowlist и всё ещё PUBLISHED. Не обещать математической проверки истинности каждого предложения малой LLM: нужен QA и консервативные отказы. Неподтверждённый ответ подавляется. Нет источников/слабый retrieval → INSUFFICIENT_EVIDENCE; LLM timeout при пригодных источниках → SEARCH_ONLY с пустым answer и ссылками. Поисковый fallback не выдаётся за генерацию.

Отзыв: DB-транзакция ставит REVOKED и увеличивает kbEpoch. Перед фактической выдачей ответа BFF под короткой блокировкой публикации повторно проверяет epoch/права; при изменении отбрасывает ответ и перечитывает источники. Если revoke завершился раньше этого commit выдачи, новые ответы/история/ссылки источник не раскрывают. Уже доставленные пользователю байты отозвать невозможно; не обещать обратного. В local-mvp один backend держит publication fence: final response/history/source read используют read-permit; revoke закрывает новые permits, фиксирует epoch, дожидается завершения или отменяет начатые публикации и только затем объявляет ENFORCED. 202 receipt сам по себе не означает ENFORCED. Долгая генерация permit не удерживает. Уже отправленные в сеть байты могут прийти позже; это не новая серверная публикация. После commit индекс чистится асинхронно. Rollback bundle не откатывает deny-реестр. Недоступный authority закрывает RAG, а не весь кредитный путь.

MVP QA: 20 golden вопросов и 10 атак с ручной проверкой каждого ответа; отдельно source recall, корректность ссылок/отказов и injection. Это smoke, не statistically validated benchmark. В target сохраняются более крупные golden/adversarial наборы, независимая оценка и lifecycle.

7. Выпуск, наблюдаемость и развитие

Локальный release bundle объединяет weights, preprocessors, feature schema, corpus/index manifest, prompt, engine versions, licences и hashes. Автор не утверждает собственный bundle: один peer проверяет evaluation digest и фиксирует approval. Перезапуск с прошлым полным bundle - rollback; файл модели отдельно от preprocessing не подменяется.

В MVP JSON logs/health/metrics endpoints и сохранённый evaluation report достаточны; запуск Grafana не является обязательной частью демо. Считать latency/errors/fallback mode, input DQ, queue age, source denials и invalid citations. Drift на малой синтетике не запускает автоматическое переобучение.

Feast, MLflow, Airflow, Evidently, Qdrant и vLLM присутствуют в логической архитектуре MVP через именованные границы и локальные реализации. Дополнительный профиль может выбирать настоящие сервисы; bank-target переводит весь стек на полноценные реализации после проверок. Их ограничения и критерии переключения описаны в общем каталоге. Training/serving пулы изолируются. Retraining требует созревших labels и data access approval; release - независимого допуска, canary и rollback. Исторические strict events меняются только новой версией схемы/темы.

Технологические адаптеры

FeatureRepositoryPort/HistoricalFeatureReader сохраняют Feast-границу; VectorIndexPort связывает NumPy/Qdrant; TextGenerationPort связывает CPU llama.cpp/vLLM. ModelRegistryPort, BatchJobRunnerPort и QualityReportPort имеют локальные рабочие реализации и будущие MLflow/Airflow/Evidently adapters. Python composition root выбирает provider; роли и immutable данные не меняются. SDK-каркас без реализации явно отклоняет вызов, а не отдаёт успешный фиктивный ответ.

Полная спецификация адаптеров; машинный каталог - contracts/technology-adapters.json из корня комплекта.